



400-650-7088

***Tong*LINK/Q-CN V10.0**

AMQP 协议部署手册

版本: 10041A01

2025 年 9 月

声明

版权

Copyright ©2025北京东方通科技股份有限公司或其关联公司（以下简称“东方通”）。保留所有权利。

版权声明

- 本文档的所有权和知识产权归属于东方通所有。
- 除非另有明确约定，东方通不授予任何明示或暗示的许可或权利，使用者不得以任何方式将本文档中的任何内容用于商业目的。
- 东方通对本文档享有版权。本文档的所有内容，包括但不限于文字、图像、图表、图标、示意图、屏幕截图等，均受版权法和国际版权条约的保护。
- 未经东方通明确授权，任何人不得对本文档以任何形式复制、修改、传播、分发、展示或进行衍生创作。

商标声明

- **东方通**、**TongTech** 标识以及其他相关东方通图形、徽标、服务名称和商标（以下统称为“东方通商标”）是东方通或其关联公司的注册商标或商标。
- 未经东方通明确授权，任何人不得以任何方式使用、复制或展示东方通商标。此外，未经东方通事先书面同意，不得将东方通商标与其他商标、标识或徽标进行混淆、链接或结合使用。
- 除非另有明确约定，本商标声明不授予任何明示或暗示的许可或权利，对东方通商标的使用须获得东方通的明确授权。
- 本文档提及的所有其他商标、标识、徽标或产品名称均为其各自所有者的财产，并可能受其各自的商标法保护。

免责声明

请在使用东方通产品之前仔细阅读本免责声明，并根据自身情况判断是否继续使用。如有任何问题，请联系我们的客户支持团队。

- 本产品的使用是基于用户自己的判断和风险评估。本文档仅作为使用指导，不对使用本产品所产生的结果做任何明示或暗示的担保。
- 东方通不对用户未按照本文档中的指导正确使用东方通产品而导致的任何损失或损害承担责任；
- 本文档可能会包含第三方提供的内容、链接或资源，这些内容由第三方自行负责。东方通对于这些内容的准确性、完整性、合法性或可靠性不承担责任。用户在使用这些内容时应自行判断和承担风险。
- 由于产品版本升级或其他原因，本文档内容会不定期更新。东方通保留在不事先通知用户的情况下随时对文档进行修改、更新或中止的权利。

如需获取有关东方通产品的许可或使用权，请联系东方通或授权代理商。任何违反本声明的行为将受到适用法律的追究。

版本变更说明

本手册的更新是累积的。因此，最新的手册版本包含对以前版本所做的所有更改。

本手册版本（10041A01）适用于TongLINK/Q-CN V10.0.4.1。

文档版本	适用产品版本	更新内容	更新日期
10041A01	V10.0.4.1	更新如下内容： 更新版权声明。	2025/9/23
10040A01	V10.0.4.0	更新如下内容： 4.2 修改配置文件：更新配置步骤。	2025/5/23
10030A01	V10.0.3.0	更新如下内容： - 第1章 概述：更新描述； 2.3 改造工作量及步骤：更新“AOP”为“AMQP协议”； 3.1 部署前准备：更新“AOP”为“AMQP协议处理程序”； 3.2 启用组件：更新“AOP”为“AMQP协议”； 3.5 验证AMQP协议端口是否正常：更新“AOP”为“AMQP协议处理程序”； 4.1 部署前准备：更新“AOP”为“AMQP协议处理程序”； 4.5 验证AMQP协议端口是否正常：更新“AOP”为“AMQP协议处理程序”； 5.2.3 打开TongLINK/Q-CN的消费端：更新“AOP”为“AMQP协议处理程序”。	2024/11/30
10020A01	V10.0.2.0	TongLINK/Q-CN V10.0.2第一次正式发布。	2024/5/31

前言

本文档是TongLINK/Q-CN V10.0产品的用户使用手册之一，提供在K8s和虚拟机上安装部署AMQP协议处理程序的基本步骤及验证方式。

阅读前注意事项

通过阅读本文档，您确认并同意自行承担因未具备必要专业背景 and 知识而导致的任何风险或后果。在使用本文档中提供的信息和指南时，请始终谨慎，并在必要时寻求专业人士建议和指导。

• 适用对象

本文档主要适用于使用本产品的系统管理员阅读，部分内容同样适用于基于本产品进行应用开发或应用部署的人员阅读。

• 专业背景

本文内容可能涉及到操作系统、服务器硬件、网络等相关领域的知识。请确保您具备相关背景 and 知识，以便更好地理解 and 应用本手册的内容。

• 技能要求

为了能够充分理解 and 应用本文档的内容，建议您具备如下技能：

1. 掌握Linux系统基本操作；
2. 掌握K8s的搭建和使用；
3. 熟悉Java开发语言。

• 术语和概念

本文档可能使用一些专业术语和概念。请确保您熟悉这些术语和概念，或者有能力查阅相关资料以便进一步理解。

• 实践经验

为了最大程度地受益于本文档，建议您具备一定实践经验。这将帮助您更好地应用文档中的操作指南 and 建议。

请注意，本文档不适用于没有相关专业背景 and 知识的用户。如果您对本文档的内容 or 所需背景有任何疑问，请在使用之前咨询相关专业人士 or 寻求额外的支持。

用户使用手册集

TongLINK/Q-CN V10.0为您提供的用户使用手册集包含的文档有：

编号	手册集文档	说明
1	000_TongLINK/Q-CN_V10.0用户手册导读	提供手册集的目录及手册版本说明。
2	001_TongLINK/Q-CN_V10.0快速使用手册	提供产品介绍，在K8s和虚拟机上安装部署产品的基本步骤及验证方式。

编号	手册集文档	说明
3	002_TongLINK/Q-CN_V10.0配置参数手册	提供配置文件及参数的详细说明。
4	003_TongLINK/Q-CN_V10.0监控管理手册	提供在K8s和虚拟机上安装部署监控管理组件的基本步骤及验证方式。
5	004_TongLINK/Q-CN_V10.0管理控制台使用手册	详细介绍管理控制台的集群管理、集群监控、告警管理和用户管理相关功能的使用说明。
6	005_TongLINK/Q-CN_V10.0命令行使用手册	详细提供产品中主要命令行的作用、相关参数及使用示例。
7	006_TongLINK/Q-CN_V10.0Java客户端开发手册	介绍使用Java客户端生产消费消息的相关概念、配置、参数及示例。
8	007_TongLINK/Q-CN_V10.0Kafka协议部署手册	提供在K8s和虚拟机上安装部署Kafka协议处理程序的基本步骤及验证方式。
9	008_TongLINK/Q-CN_V10.0AMQP协议部署手册	提供在K8s和虚拟机上安装部署AMQP协议处理程序的基本步骤及验证方式。
10	009_TongLINK/Q-CN_V10.0JMS客户端开发手册	介绍如何将适配ActiveMQ的JMS客户端迁移为适配TongLINK/Q-CN的JMS客户端。
11	010_TongLINK/Q-CN_V10.0MQTT协议部署手册	提供在K8s和虚拟机上安装部署MQTT协议处理程序的基本步骤及验证方式。
12	011_TongLINK/Q-CN_V10.0服务端进阶配置指南	提供在K8s和虚拟机上开启TongLINK/Q-CN的国密通信、消息轨迹、消息过滤等高级功能特性的方法。
13	012_TongLINK/Q-CN_V10.0灾备及故障转移手册	介绍使用灾备及故障转移的相关概念、配置、参数及示例。

技术支持

东方通产品将为您提供全方位的技术支持，您可以通过以下方式获得技术支持：

- 网址：www.tongtech.com
- 电话：400-650-7088
- 邮箱：support@tongtech.com

您在取得技术支持时，请提供如下信息：

- 您的姓名

- 您的公司信息
- 您的联系方式
- 操作系统及其版本
- 产品版本号
- 日志等错误的详细信息

目录

声明	1
版本变更说明	2
前言	3
1. 概述	7
2. 注意事项	8
2.1. RabbitMQ版本支持	8
2.2. 支持RabbitMQ的功能及限制	8
2.3. 改造工作量及步骤	8
2.4. 使用建议	9
3. K8s部署	10
3.1. 部署前准备	10
3.2. 启用组件	10
3.3. 集群部署更新	11
3.4. 验证组件是否部署成功	11
3.5. 验证AMQP协议端口是否正常	11
4. 虚拟机部署	13
4.1. 部署前准备	13
4.2. 修改配置文件	13
4.3. 检查插件包	14
4.4. 重启broker	14
4.5. 验证AMQP协议端口是否正常	14
5. AMQP协议业务验证	15
5.1. 在TongLINK/Q-CN上为vhost添加namespace	15
5.1.1. K8s环境	15
5.1.2. 虚拟机环境	15
5.2. 使用Java代码客户端验证AMQP基本功能	15
5.2.1. 添加依赖包	15
5.2.2. 编写和运行RabbitMQ的生产和消费Java代码	16
5.2.3. 打开TongLINK/Q-CN的消费端	19
附录 A: 术语说明	21

1. 概述

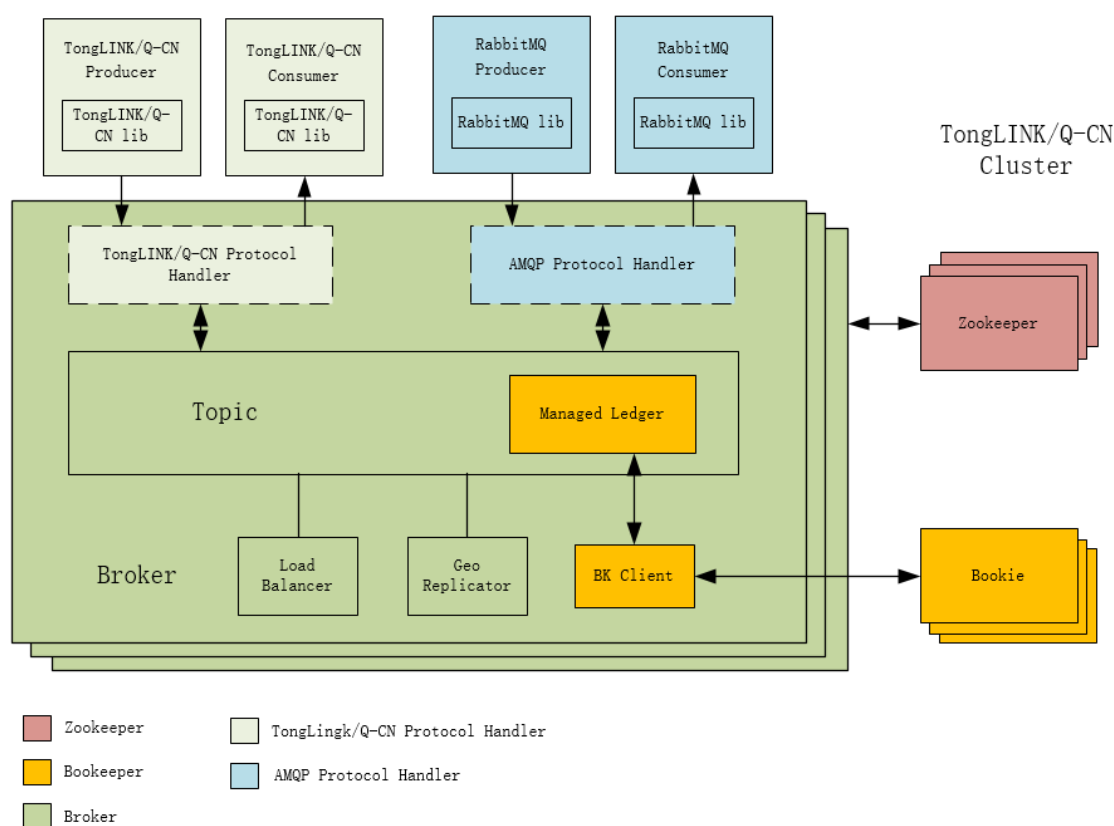
通过在TongLINK/Q-CN的broker上引入一个AMQP协议处理程序，使TongLINK/Q-CN支持AMQP协议。通过向现有的TongLINK/Q-CN集群添加AMQP协议处理程序，可以在不修改现有RabbitMQ客户端的情况下，服务端迁移到TongLINK/Q-CN。这使得RabbitMQ应用程序也能够利用TongLINK/Q-CN的强大功能，例如：

1. 具备云原生、企业级多租户，万级topic等能力；
2. 具备无重新平衡体系结构的简化操作，横向快速动态扩展的优势；
3. 利用BookKeeper和分层存储实现无限事件流保留和海量数据堆积；
4. 利用Functions实现无服务器事件处理。

AMQP协议处理程序作为TongLINK/Q-CN协议处理程序的插件，在TongLINK/Q-CN的Broker启动时加载。它通过在TongLINK/Q-CN上提供本机的AMQP0-9-1协议客户端支持（典型的是RabbitMQ客户端），帮助降低RabbitMQ客户端在使用TongLINK/Q-CN集群时的障碍，从而帮助用户成功迁移业务系统。

AMQP协议处理程序通过利用TongLINK/Q-CN已有的组件，在TongLINK/Q-CN上实现了AMQP 0-9-1传输协议到TongLINK/Q-CN（Pulsar）协议的转换。

以下图示说明了AMQP协议处理程序在TongLINK/Q-CN中的实现方式。



2. 注意事项

2.1. RabbitMQ版本支持

支持AMQP 0-9-1版本的协议，暂不支持AMQP 1.0版本的协议。

2.2. 支持RabbitMQ的功能及限制

- **支持的功能：**
 - 无交换机的简单模式（hello world）和工作模式（Work queues）；
 - 有交换机的直连交换机(Direct exchange)，扇形交换机(Fanout exchange)，和主题交换机(Topic exchange)；
 - 只支持持久化（durable）的交换机和持久化（durable）的队列。
- **不支持的功能：**
 - 消息优先级；
 - 鉴权；
 - 授权；
 - 事务；
 - SSL加密；
 - DLQ（死信队列）；
 - 延时消息；
 - TTL。
- **TongLINK/Q-CN侧限制：**

一个RabbitMQ的vhost对应TongLINK/Q-CN中的单bundle的namespace（该namespace下只有一个bundle）。
- **关闭开关：**

事务功能关闭，transactionCoordinatorEnabled: false。

2.3. 改造工作量及步骤

1. 服务端部署为TongLINK/Q-CN服务端，参见《001_TongLINK/Q-CN_V10.0快速使用手册》，并部署监控、控制台等组件，参见《003_TongLINK/Q-CN_V10.0监控管理手册》。
2. 使用TongLINK/Q-CN的客户端进行生产和消费数据。
3. （可选）部署TongLINK/Q-CN服务端支持AMQP协议功能，请参考本手册[K8s环境部署](#)或[虚拟机环境部署](#)。

注意： 如果不使用RabbitMQ的客户端，此步骤可以省略。

4. （可选）客户端更改目标服务端的IP地址和端口为TongLINK/Q-CN的服务端。

注意：

- 如果不使用RabbitMQ的客户端，此步骤可以省略；
- 尽量优先使用TongLINK/Q-CN的客户端，尽量少使用RabbitMQ的客户端。

5. （可选）服务端历史数据迁移到TongLINK/Q-CN中。

注意：

- 如果不需要把RabbitMQ中的历史数据转移到TongLINK/Q-CN中，此步骤可省略；
- RabbitMQ服务器上的历史数据可以等处理完毕归档或数据过期后归档。

2.4. 使用建议

使用RabbitMQ客户端的生产和消费，是用于解决RabbitMQ客户端因各种客观原因无法更换为TongLINK/Q-CN客户端的问题。在实现过程中，协议的转换会降低少部分效率（损耗远小于使用TongLINK/Q-CN的服务端替换RabbitMQ带来的效率提升），且存在少量两种协议兼容性的测试覆盖考虑不周的问题，因此，厂家建议，如果能更换TongLINK/Q-CN的客户端，尽量更换为TongLINK/Q-CN的客户端。

3. K8s部署

3.1. 部署前准备

在K8s上部署TongLINK/Q-CN的AMQP协议处理程序前，请先按照《001_TongLINK/Q-CN_V10.0快速使用手册》部署TongLINK/Q-CN集群并验证成功。

3.2. 启用组件

1. 进入tonglinkq-cn目录。

```
cd tonglinkq-cn
```

2. 修改values.yaml中的如下配置（粗体为需修改项），将components.aop的值从false（默认值）修改为true以启用组件；根据需要修改AMQP协议对外暴露的基础端口号（aop.service.proxyNodePort）；关闭事务开关，将aop.PULSAR_PREFIX_transactionCoordinatorEnabled的值从true修改为false：

```
vi values.yaml
```

```
components:

  aop: false

  ...

  ## Pulsar: AoP Protocol Handler

  aop:

    ports:

      plaintext: 5672

      proxy: 5682

    service:

      proxyNodePort: 15682

    ...

    PULSAR_PREFIX_transactionCoordinatorEnabled: "true"
```

3.3. 集群部署更新

- 进入tonglinkq-cn目录后，执行如下命令更新部署TongLINK/Q-CN：

```
helm upgrade test -n tlq-cn ./
```

其中test为helm部署的release名称；-n为命名空间名称，注意与已部署集群的helm部署一致。

- （可选）如需删除集群，使用如下命令：

```
helm uninstall test -n tlq-cn
```

3.4. 验证组件是否部署成功

在K8s服务器的master节点上执行如下命令查看集群各组件运行状态：

```
kubectl get pods -n <namespace>
```

其中-n为命名空间名称，本例中为tlq-cn。

如下图所示，结果中tonglinkq-console, tonglinkq-prometheus, tonglinkq-grafana, tonglinkq-alert-manager, tonglinkq-node-exporter都为running状态，则部署更新成功。

```
[root@k8s-master tong-helm-chart]# kubectl get pods -n tlq-cn
NAME                                READY   STATUS    RESTARTS   AGE
test-tonglinkq-alert-manager-0      2/2    Running   0           5m11s
test-tonglinkq-bookie-0             1/1    Running   0           5m12s
test-tonglinkq-bookie-1             1/1    Running   0           5m11s
test-tonglinkq-bookie-2             1/1    Running   0           5m11s
test-tonglinkq-bookie-init-p8nmr    0/1    Completed 0           5m12s
test-tonglinkq-broker-0             1/1    Running   0           5m12s
test-tonglinkq-cluster-init-g84dp   0/1    Completed 0           5m12s
test-tonglinkq-console-0            1/1    Running   0           5m12s
test-tonglinkq-console-init-b5wb4   0/1    Completed 0           5m12s
test-tonglinkq-detector-0           1/1    Running   0           5m12s
test-tonglinkq-grafana-0            1/1    Running   0           5m11s
test-tonglinkq-node-exporter-lcrn8  1/1    Running   0           5m12s
test-tonglinkq-node-exporter-qg49h  1/1    Running   0           5m12s
test-tonglinkq-prometheus-0         2/2    Running   0           5m12s
test-tonglinkq-proxy-0              1/1    Running   0           5m12s
test-tonglinkq-recovery-0           1/1    Running   0           5m11s
test-tonglinkq-toolset-0            1/1    Running   0           5m12s
test-tonglinkq-zookeeper-0          1/1    Running   0           5m12s
test-tonglinkq-zookeeper-1          1/1    Running   0           5m11s
test-tonglinkq-zookeeper-2          1/1    Running   0           5m11s
[root@k8s-master tong-helm-chart]#
```

3.5. 验证AMQP协议端口是否正常

使用如下命令查看端口是否被暴露：

```
kubectl get svc -n <namesapce> | grep tonglinkq-broker
```

暴露的节点端口与broker数量一致，如下图所示：

```
[root@k8s-master zyj]# kubectl get svc -n test |grep aop
test-tonglinkq-broker-aop NodePort 10.110.134.128 <none> 5682:15682/TCP 57m
[root@k8s-master zyj]#
```

图示中端口为15682，即是5682对外暴露的AMQP协议端口。

如果端口正常，请跳转到[AMQP协议业务验证](#)。

4. 虚拟机部署

4.1. 部署前准备

在虚拟机上部署TongLINK/Q-CN的AMQP协议处理程序前，请先按照《001_TongLINK/Q-CN_V10.0快速使用手册》部署TongLINK/Q-CN集群并验证成功。

注意：如果是集群部署，则每个节点上都需要进行如下4.2 ~ 4.4章节的配置。

4.2. 修改配置文件

1. 进入broker配置文件目录/home/broker/conf

```
cd /home/broker/conf
```

2. 修改broker.conf配置文件。

```
vi broker.conf
```

具体配置如下：

```
...

messagingProtocols=amqp    # 协议

protocolHandlerDirectory=./protocols    # 协议插件存放路径

amqpListeners=amqp://192.148.0.x:5672    # 192.148.0.x为服务端内网ip地址

transactionCoordinatorEnabled=false    # 关闭事务消息功能
```

3. 在TLQ-CN目录执行以下命令更新broker-child.conf内容到broker.conf文件：

```
./conf/update_config.sh broker-child.conf broker.conf
```

4. （可选）开启日志的debug模式：

修改conf/log4j2.yaml文件，示例如下：

```
Logger:

- name: io.streamnative.pulsar.handlers.amqp

  level: debug

  additivity: false
```

AppenderRef:

- ref: Console

4.3. 检查插件包

在TongLINK/Q-CN安装目录下，进入protocols文件夹，查看是否有对应版本的nar包，如下所示：

```
[xxx@master1 protocols]# ll

-r----- 1 root root 6415389 11月 30 17:34 pulsar-protocol-handler-amqp-3.0.0.4.nar

-r----- 1 root root 54477772 10月 25 11:02 pulsar-protocol-handler-kafka-3.0.0.4.nar
```

4.4. 重启broker

集群部署的每一台broker的配置都需要修改，同时也需要重启。请参考《001_TongLINK/Q-CN_V10.0快速使用手册》中的“启停broker”章节，先停止broker，再启动broker。

4.5. 验证AMQP协议端口是否正常

使用netstat -lntp | grep 5672，查看端口是否被监听：

```
[root@k8s-node2 ~]# netstat -lntp | grep 5672
tcp        0      0 0.0.0.0:5672          0.0.0.0:*            LISTEN     5837/java
[root@k8s-node2 ~]#
```

如果端口正常，请跳转到[AMQP协议业务验证](#)。

5. AMQP协议业务验证

5.1. 在TongLINK/Q-CN上为vhost添加namespace

5.1.1. K8s环境

进入toolset容器，执行如下命令：

```
kubect exec -it <toolset_pod> -n <namespace> -- bash
```

如下图所示：

```
[root@k8s-master tong-helm-chart]# kubectl exec -it test-tonglinkq-toolset-0 -n test -- bash
I have no name!@test-tonglinkq-toolset-0:/pulsar$
I have no name!@test-tonglinkq-toolset-0:/pulsar$
```

例如，vhost名称为“vhost1”，则租户和命名空间为：public/vhost1

```
./bin/tong-admin namespaces create -b 1 public/vhost1
```

为namespace设置消息持久化，如保存100M或者是2天

```
./bin/tong-admin namespaces set-retention -s 100M -t 2d public/vhost1
```

5.1.2. 虚拟机环境

例如，vhost名称为“vhost1”，则租户和命名空间为：public/vhost1

```
./bin/tong-admin namespaces create -b 1 public/vhost1
```

为namespace设置消息持久化，如保存100M或者是2天

```
./bin/tong-admin namespaces set-retention -s 100M -t 2d public/vhost1
```

5.2. 使用Java代码客户端验证AMQP基本功能

5.2.1. 添加依赖包

在Maven / Gradle项目中添加对应版本的依赖包：

```
<dependency>

  <groupId>com.rabbitmq</groupId>

  <artifactId>amqp-client</artifactId>

  <version>5.8.0</version>
```



```
</dependency>
```

如果使用Gradle，添加依赖如下：

```
dependencies {

    compile 'com.rabbitmq:amqp-client:5.8.0'

}
```

版本要求：

- Java客户端具体版本号可参考：<https://repo1.maven.org/maven2/com/rabbitmq/amqp-client/>
- 5.X版本支持JDK8及以上
- 4.X版本支持JDK6及以上

5.2.2. 编写和运行RabbitMQ的生产和消费Java代码

代码示例如下：

```
// 导入包

import com.rabbitmq.client.AMQP;

import com.rabbitmq.client.BuiltinExchangeType;

import com.rabbitmq.client.Channel;

import com.rabbitmq.client.Connection;

import com.rabbitmq.client.ConnectionFactory;

import com.rabbitmq.client.DefaultConsumer;

import com.rabbitmq.client.Envelope;

import java.io.IOException;

import java.util.concurrent.CountDownLatch;

import java.util.concurrent.TimeoutException;

// 类
```

```

public class SampleAoPTest {

    public static void main(String[] args) throws IOException, TimeoutException, InterruptedException{

// 创建连接

        ConnectionFactory connectionFactory = new ConnectionFactory();

        connectionFactory.setVirtualHost("vhost1"); // vhost

        connectionFactory.setHost("127.0.0.1"); // 服务端ip地址

        connectionFactory.setPort(5672); // 服务端amqp端口

        Connection connection = connectionFactory.newConnection();

        Channel channel = connection.createChannel();

        String exchange = "ex";

        String queue = "qu";

// 交换机声明

        channel.exchangeDeclare(exchange, BuiltinExchangeType.FANOUT, true, false, false, null);

// 队列声明和绑定

        channel.queueDeclare(queue, true, false, false, null);

        channel.queueBind(queue, exchange, ""); //

// 发布消息

        for (int i = 0; i < 10; i++) {

            channel.basicPublish(exchange, "", null, ("hello - " + i).getBytes());

        }

// 消费消息

        CountDownLatch countDownLatch = new CountDownLatch(100);
    
```

```
channel.basicConsume(queue, true, new DefaultConsumer(channel) {

    @Override

    public void handleDelivery(String consumerTag, Envelope envelope, AMQP.BasicProperties properties,
byte[] body) throws IOException {

        System.out.println("receive msg: " + new String(body));

        countDownLatch.countDown();

    }

});

countDownLatch.await();

// 释放资源

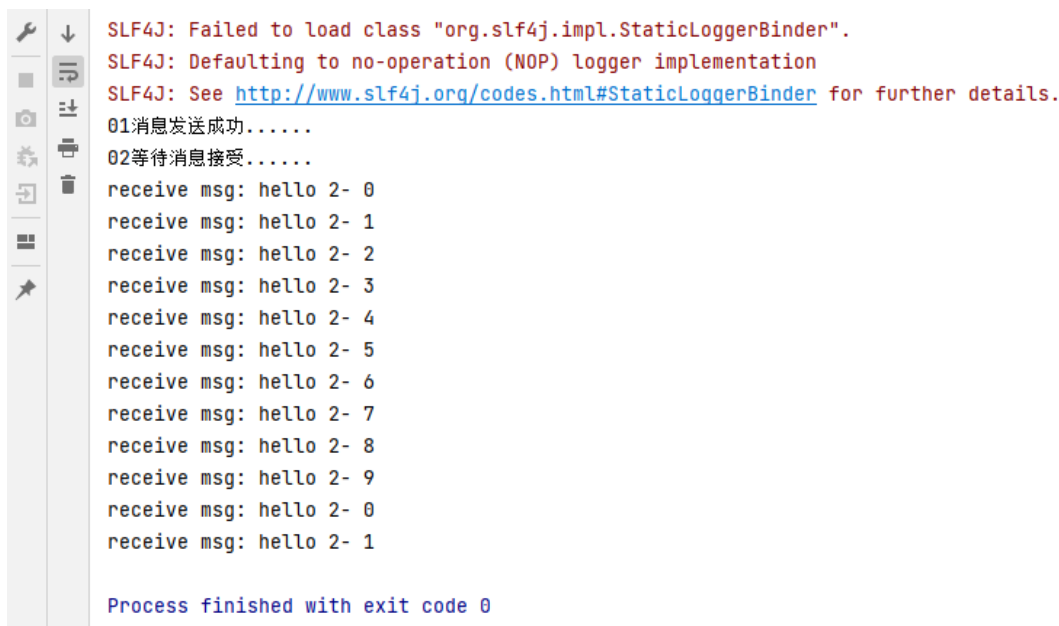
channel.close();

connection.close();

}

}
```

运行后结果如下:



```
SLF4J: Failed to load class "org.slf4j.impl.StaticLoggerBinder".
SLF4J: Defaulting to no-operation (NOP) logger implementation
SLF4J: See http://www.slf4j.org/codes.html#StaticLoggerBinder for further details.
01消息发送成功.....
02等待消息接受.....
receive msg: hello 2- 0
receive msg: hello 2- 1
receive msg: hello 2- 2
receive msg: hello 2- 3
receive msg: hello 2- 4
receive msg: hello 2- 5
receive msg: hello 2- 6
receive msg: hello 2- 7
receive msg: hello 2- 8
receive msg: hello 2- 9
receive msg: hello 2- 0
receive msg: hello 2- 1

Process finished with exit code 0
```

结论: 表示可以通过RabbitMQ的客户端生产和消费。

5.2.3. 打开TongLINK/Q-CN的消费端

租户为public，命名空间为vhost1，会自动产生两个主题：

persistent://public/vhost1/__amqp_queue__qu #用于存放路由到这个队列的索引消息。

persistent://public/vhost1/__amqp_exchange__ex #用于存储实际数据。

- 如果使用虚拟机部署，请使用TongLINK/Q-CN消费端如下消费命令。
- 如果使用K8s客户端，请参考[K8s环境](#)进入toolset容器，执行如下命令。

./bin/tong-client consume persistent://public/vhost1/__amqp_exchange__ex -s my-sub -n 0

```
[root@k8s-node2 apache-pulsar-3.0.0]# ./bin/tong-client consume
persistent://public/vhost1/__amqp_exchange__ex -s my-sub -n 0
... ..
----- got message -----
key:[null], properties:[_pi_mandatory=false, _pi_exchange=ex, _bp_timestamp=0, _bp_delivery_mode=,
_bp_priority=, _pi_immediate=false, _bp_property_flags=0, _bp_expiration=0], content:hello - 0
----- got message -----
key:[null], properties:[_pi_mandatory=false, _pi_exchange=ex, _bp_timestamp=0, _bp_delivery_mode=,
_bp_priority=, _pi_immediate=false, _bp_property_flags=0, _bp_expiration=0], content:hello - 1
----- got message -----
key:[null], properties:[_pi_mandatory=false, _pi_exchange=ex, _bp_timestamp=0, _bp_delivery_mode=,
_bp_priority=, _pi_immediate=false, _bp_property_flags=0, _bp_expiration=0], content:hello - 2
----- got message -----
key:[null], properties:[_pi_mandatory=false, _pi_exchange=ex, _bp_timestamp=0, _bp_delivery_mode=,
_bp_priority=, _pi_immediate=false, _bp_property_flags=0, _bp_expiration=0], content:hello - 3
----- got message -----
key:[null], properties:[_pi_mandatory=false, _pi_exchange=ex, _bp_timestamp=0, _bp_delivery_mode=,
_bp_priority=, _pi_immediate=false, _bp_property_flags=0, _bp_expiration=0], content:hello - 4
----- got message -----
key:[null], properties:[_pi_mandatory=false, _pi_exchange=ex, _bp_timestamp=0, _bp_delivery_mode=,
_bp_priority=, _pi_immediate=false, _bp_property_flags=0, _bp_expiration=0], content:hello - 5
----- got message -----
key:[null], properties:[_pi_mandatory=false, _pi_exchange=ex, _bp_timestamp=0, _bp_delivery_mode=,
_bp_priority=, _pi_immediate=false, _bp_property_flags=0, _bp_expiration=0], content:hello - 6
----- got message -----
key:[null], properties:[_pi_mandatory=false, _pi_exchange=ex, _bp_timestamp=0, _bp_delivery_mode=,
_bp_priority=, _pi_immediate=false, _bp_property_flags=0, _bp_expiration=0], content:hello - 7
----- got message -----
key:[null], properties:[_pi_mandatory=false, _pi_exchange=ex, _bp_timestamp=0, _bp_delivery_mode=,
_bp_priority=, _pi_immediate=false, _bp_property_flags=0, _bp_expiration=0], content:hello - 8
----- got message -----
key:[null], properties:[_pi_mandatory=false, _pi_exchange=ex, _bp_timestamp=0, _bp_delivery_mode=,
_bp_priority=, _pi_immediate=false, _bp_property_flags=0, _bp_expiration=0], content:hello - 9
```

结论：通过RabbitMQ客户端生产的数据，可以通过TongLINK/Q-CN的客户端和RabbitMQ客户端同时消费成功。AMQP协议处理程序基本功能验证成功。

附录 A: 术语说明

名称	说明
TongLINK/Q-CN (简称TLQ-CN)	东方通云原生消息中间件
Broker	服务端工作节点
Client	客户端
Topic	主题
Message	消息
Producer	生产者，即消息的提供者
Consumer	消费者，即消息的消费者
Pub-Sub	发布及订阅
Point-to-Point(PTP)	点对点模式，即生产者和消费者之间的消息往来

